Problem : Generating the video from text.

Use Python Version : 3.11
Generated Video automatically saves in the main directory.
Video dataset also downloaded in main directory.

Problem Fit & Scope :

The problem fits well with **parameter-efficient fine-tuning methods such as LoRA** because:

- Training full video diffusion models requires **very large GPU memory and datasets**
- LoRA allows **lightweight adaptation of pretrained models**
- Small video datasets can still be useful when combined with **pretrained motion priors**

Using frameworks like **PyTorch, Diffusers, and PEFT**, the system can efficiently adapt models like:

- AnimateDiff
- Stable Video Diffusion (SVD)
- ModelScope Text-to-Video

This approach makes **video generation research feasible on limited hardware (e.g., Mac with fp16 and gradient checkpointing).**

Data Preparation

- Collecting 100-300 short video clips (2-4 seconds)
- Generating captions using BLIP
- Cleaning captions manually

Model Training

- Fine-tuning video diffusion models using LoRA
- Training configuration:
    - resolution: 256×256
    - frames: 16
    - batch size: 1
    - training steps: 1k–5k

Experiments

- Compare architectures:

  AnimateDiff
  Stable Video Diffusion
  ModelScope

- Test different LoRA ranks (4, 8, 16)
- Evaluate motion LoRA vs full LoRA

Evaluation Metrics

- CLIPScore ≥ 0.25 (text-video alignment)
- tLPIPS ≤ 0.30 (temporal consistency)

Data Acquisition & Quality :

```
from huggingface_hub import snapshot_download

snapshot_download(
```

```
        repo_id="Wild-Heart/Disney-VideoGeneration-Dataset",
        repo_type="dataset",
        local_dir="./video_dataset"
)
```

Videos : 69 Video (video_dataset / videos /)
Captions : 69 prompt (video_dataset / prompt.txt)

Baseline & Experiments :

Baseline Model for this Project is Stable Video Diffusion (SVD) using the pretrained weights from the huggingFace diffusers library.

The baseline setup uses the original pretrained model without any finetunning.
This allows us to evaluate how well the model performs before applying LoRA-based fine tuning.

Parameter	Value
Resolution	256 x 256
Batch Size	1
Frames	16
Training Steps	0 (no training)
Scheduler	DDPM
Model	Stable Video Diffusion

Conditioning	First Frame+ Prompt

BaseLine WorkFlow:

Input Prompt
|
Stable Diffusion (text->image)
|
Initial Image
|
Stable Video Diffusion
|
Generated Video

FineTunning Setup:

To improve video quality and prompt alignment the model is finetunning using (LoRA).
LoRA enables efficient training by updating only a small number of parameters keeping the original model weights frozen.

Parameter	Value
Resolution	256 x 256
Batch Size	1
Frames	16
Training Steps	1k-5k
Scheduler	DDPM
Model	Stable Video Diffusion

Conditioning	First Frame+ Prompt
optimizer	AdamW
Learning Rate	1e-4
Precision	FP16
Acceleration	Gradient CheckPointing

Experiments:

1. LoRA rank comparison.

Rank	Expected Behaviour
4	LightWeight training but limited adaption
8	Balanced Performance
16	Higher flexibility but increase memory usage.

2. Training Steps

Steps	Purpose
1000	Initial Adaption
2000	Improved motion consistency
3000	Better prompt alignment.

3. Video Resolution

Resolution	Effect
256 x 256	Faster training , lower memory
512 x 512	Higher visual quality but expensive

Training Correctness and Efficiency :

1. Correct Training Objective: Model is trained using the diffusion prediction objective. UNet learns to predict the noise component.

   Training Objective:

   Loss = MSE(predicted_noise,true_noise)

2. Correct Input Representation: Training pipeline that inputs follow the format required by **Stable VideoDiffusion (SVD).**

   Video Latents :

   The full video is encoded into latent representation using the VAE encoder.

   Video Frames -> Variational Encoder (VAE) -> Video Latents.

   Conditioning Latents : The first frame is separately encoded to guide video generation.

   First Frame -> VAE Encoder -> Conditioning Latents.

   The Unet Receives:

8 channels

4 -> Noise video latents

4-> First-frame conditioning latents

3. LoRA Parameter Training:

    Original UNet weights are Frozen.

    Only LoRA layers are trainable.

    Lora Configuration :

        Rank : 4

        Alpha: 8

        Initialization : gaussian

4. Gradient Stability:

    Gradient checkpoint is enabled to reduce memory usage.

    Mixed precision training (FP16) is used.

    AdamW optimizer ensures stable parameter updates.

    Optimizer Configurations:

        Optimizer : AdamW

        Learning rate: 1e-4

        Parameters: LoRA weights

5. Efficient Data Pipeline:

    Video frames resized to 256 x 256

    Frames normalized to [0,1]

    Tensor format converted to : [frames,channles,height,width]

    Pytorch dataloader handles batching and shuffling.

Evaluation & Metrics :

Evaluation focuses on three key aspects:

Text -to - Video alignment

Visual Quality

Temporal Consistency

1. CLIPScore (Text-Video Alignment):

    CLIPScore measures how well the generated video matches the input text prompt.

    This CLIP Model computes similarity between: text embeddings and visual embeddings extracted from video frames.

Extracted Frames from the generated Video

|

Encode frames using the CLIP image encoder

|

Encode the text prompt using the CLIP text encoder

|

Compute cosine similarity

2. Temporal LPIPS :

It measures temporal consistency between consecutive frames.

It evaluates whether objects:

Remain stable

Move naturally

Maintain color and shape across frames.

tLPIPS = mean (LPIPS (frames_t,frames_t+1))

3. Frame level LPIPS (perceptual similarity):

LPIPS (learned perceptual image patch similarity) measure visual quality differences between frames.

4. Frame Consistency Score:

It evaluates whether objects maintain consistent structure across frames.

Generated Video

|

Frame Extraction

|

CLIP Embeddings

|

CLIP score calculation

|

TLPIPS Calculation

|

Human Evaluation

Minimal UI / Demo for Data Acquisition and Inference :

The UI allows users to:

1. Input a text prompt
2. Generate an image using Stable Diffusion
3. Convert the image into a short video Stable Video Diffusion

4. Display in Gradio.

Data Acquisition:

Example dataset entry:

```
{
    “Video_path”: “video_dataset/videos/id.mp4’,
    “Caption”: “ some prompt”,
}
```

Success Criteria & Insight :

```
Generating initial image
100%|██████████| 50/50 [00:55<00:00,  1.11s/it]
Pipelines loaded with `dtype=torch.float16` cannot run with `cpu` device. It is not recommended to move them to `cpu` as running them will fail. Please make su
Pipelines loaded with `dtype=torch.float16` cannot run with `cpu` device. It is not recommended to move them to `cpu` as running them will fail. Please make su
Pipelines loaded with `dtype=torch.float16` cannot run with `cpu` device. It is not recommended to move them to `cpu` as running them will fail. Please make su
Pipelines loaded with `dtype=torch.float16` cannot run with `cpu` device. It is not recommended to move them to `cpu` as running them will fail. Please make su
Generating video frames
100%|██████████| 25/25 [25:37<00:00, 61.48s/it]
Pipelines loaded with `dtype=torch.float16` cannot run with `cpu` device. It is not recommended to move them to `cpu` as running them will fail. Please make su
Pipelines loaded with `dtype=torch.float16` cannot run with `cpu` device. It is not recommended to move them to `cpu` as running them will fail. Please make su
Pipelines loaded with `dtype=torch.float16` cannot run with `cpu` device. It is not recommended to move them to `cpu` as running them will fail. Please make su
Saving video
Python(15942) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.
Python(15943) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.
Python(15944) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.
Python(15945) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.
Python(15946) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.
/Users/pagupta/Desktop/GD_Projects/module_2_all_courses/ML_Specialization_coursera/ml_special_311/lib/python3.11/site-packages/open_clip/factory.py:450: UserWa
  warnings.warn(
Python(16167) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.
CLIPScore: 0.2571637239307165
```

CLIP score is : 0.257163 >=0.25

Higher values indicate stronger alignment between the prompt and the generated video.

```
Setting up [LPIPS] perceptual loss: trunk [alex], v[0.1], spatial [off]
Downloading: "https://download.pytorch.org/models/alexnet-owt-7be5be79.pth" to /Users/pagupta/.cache/torch/hub/checkpoints/alexnet-owt-7be5be79.pth
100%|██████████| 233M/233M [01:42<00:00, 2.40MB/s]
Loading model from: /Users/pagupta/Desktop/GD_Projects/module_2_all_courses/ML_Specialization_coursera/ml_special_311/lib/python3.11/site-packages/lpips/we
Python(18471) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.
CLIPScore: 0.25397221557796
tLPIPS: 0.04083861426879016
```

tLPIPS score: 0.0408 <=0.30

Lower values indicate smoother transitions and better temporal coherence.

Reproducibility & Docs :

Create Virtual Environment:

python3 -m venv env_name
source venv/bin/activate
pip install --upgrade pip
and then run the following command to install dependencies
pip install -r requirements.txt

requirement.txt url:
https://github.com/Pawanguptapawan/GD_Carriculum/blob/main/requirements.txt

Github URL:
https://github.com/Pawanguptapawan/GD_Carriculum/blob/main/FineTunning_with_LoRA_config_Text_to_Video.ipynb